Introduction aux LLMs

Séance 4 : Utilisation et intégration d'outils

ESGI Paris

Célia Nouri · `celia.nouri@inria.fr`

Semestre 1, 2026–2027

Agenda

1. Évaluation
2. Prompting : zero-shot, few-shot, Chain-of-Thought
3. Contraintes d'inférence : fenêtre de contexte, quantisation, coût
4. Retrieval-Augmented Generation (RAG)
5. Appel d'outils et function calling (Toolformer)
6. Lab 4 : Prompting et RAG

1. Évaluation

Pourquoi évaluer un LLM est difficile

- Un LLM est **généraliste** : pas de tâche unique à mesurer
- Les benchmarks se **saturent** vite (un nouveau modèle bat le précédent record en quelques mois)
- Risque de **contamination** : le benchmark peut avoir fuité dans les données de pré-entraînement (section 2)
- Un bon score sur un benchmark ne garantit pas une bonne expérience utilisateur réelle

Benchmarks académiques classiques

Benchmark	Mesure	Format
MMLU (Hendrycks et al., 2020)	Connaissances générales, 57 matières	QCM
HellaSwag (Zellers et al., 2019)	Sens commun, complétion de phrase plausible, mauvaises réponses construites par **filtrage adversarial** (*Adversarial Filtering*)	QCM
GSM8K (Cobbe et al., 2021)	Raisonnement mathématique (niveau collège)	Problèmes ouverts
HumanEval (Chen et al., 2021)	Génération de code fonctionnel	*pass@k* (le code s'exécute-t-il ?)

Ces benchmarks apparaissent systématiquement dans les *model cards* (GPT-4, Claude, LLaMA, Gemini…) pour comparer les modèles.

Deux précisions sur ces benchmarks

HellaSwag n'est pas juste "difficile par hasard" : ses mauvaises réponses sont construites par **filtrage adversarial** (*Adversarial Filtering*) — un modèle génère plusieurs fins de phrase plausibles, on ne garde que celles qui **trompent des modèles** tout en restant **évidentes pour un humain**. Le benchmark est donc conçu spécifiquement pour être dur pour les machines.

HumanEval ne veut pas dire "évalué par des humains" : le nom vient du fait que les **164 problèmes de programmation sont écrits à la main** par des humains (énoncé + tests unitaires). L'évaluation elle-même est **automatique** : le code généré est exécuté contre les tests, et *pass@k* mesure la proportion de problèmes résolus, aucun jugement humain n'intervient dans la note.

Le problème de la contamination

Les modèles sont entraînés sur **une grande partie du web** qui contient parfois les benchmarks eux-mêmes, ou leurs solutions.

```
MMLU publié (2020) → corrigés/fiches de révision postés sur GitHub, Quizlet...
                   → ces pages indexées et présentes dans Common Crawl (2022+)
                   → potentiellement vues pendant le pré-entraînement d'un modèle
                   → score artificiellement élevé au moment de l'évaluation
```

Conséquence : un score MMLU élevé peut refléter de la **mémorisation** plutôt que de la **généralisation**. D'où l'intérêt de benchmarks récents, privés, ou renouvelés régulièrement.

Comparaison des LLMs

Face aux limites des QCM, deux approches complémentaires :

Chatbot Arena (LMSYS, 2023) : des humains comparent en aveugle les réponses de deux modèles à un même prompt et votent pour la meilleure → classement de type **Elo** (comme aux échecs).

- **Rappel — le classement Elo** : chaque modèle a un score. Avant un match, on calcule sa **probabilité attendue de gagner** à partir de l'écart de score avec l'adversaire. Après le match, son score est ajusté : gagner contre un modèle **mieux classé** rapporte plus de points que gagner contre un modèle plus faible (et l'inverse pour une défaite).

LLM-as-judge

LLM-as-judge (MT-Bench, Zheng et al., 2023) : on utilise un LLM puissant (GPT-4, Claude) pour **noter automatiquement** les réponses d'autres modèles selon une grille de critères explicités dans le prompt.

⚠️ **Zheng et al. (2023)**, dans ce même papier MT-Bench, mesurent et documentent trois biais du LLM-as-judge : préférence pour les réponses **longues** (*verbosity bias*), pour son **propre style** de génération (*self-enhancement bias*), et sensibilité à l'**ordre de présentation** des réponses comparées (*position bias*).

🧩 Quiz — Évaluation

1. Pourquoi un score élevé sur MMLU ne garantit-il pas qu'un modèle est réellement meilleur ?

2. Quelle est la différence entre l'évaluation Chatbot Arena et un benchmark comme HumanEval ?

3. Citez un biais connu du LLM-as-judge.

🧩 Quiz — Réponses

1. Risque de **contamination** : le benchmark (ou ses réponses) peut avoir été vu pendant le pré-entraînement, gonflant artificiellement le score sans réelle généralisation.

2. Chatbot Arena repose sur des **comparaisons humaines en aveugle** avec classement Elo ; HumanEval est un test **automatique** (le code s'exécute-t-il correctement ?).

3. Préférence pour les réponses longues, préférence pour son propre style de génération, ou sensibilité à l'ordre de présentation des réponses comparées.

Ressources — Évaluation

📄 **MT-Bench / Chatbot Arena** : Zheng et al. (2023); arxiv.org/abs/2306.05685

📖 **LMSYS Chatbot Arena** : arena.ai

2. Prompting

Le prompt comme levier

Une fois le modèle pré-entraîné, instruction-tuné et aligné (séance 3), on ne le **ré-entraîne pas** à chaque nouvel usage.

Le **prompt** — le texte qu'on envoie au modèle — devient le principal levier pour changer son comportement, **sans toucher aux poids**.

```
Fine-tuning / RLHF  : modifie les poids du modèle          (coûteux, lent)
Prompting           : modifie seulement l'entrée du modèle (gratuit, instantané)
```

C'est ce qu'on appelle l'**apprentissage en contexte** (*in-context learning*).

Zero-shot prompting

Le modèle reçoit uniquement une **instruction**, sans aucun exemple de ce qu'on attend.

```
Prompt :
"Classe ce message parmi {Spam, Non-spam} :
 'Félicitations ! Vous avez gagné un iPhone, cliquez ici pour le réclamer.'"

Réponse attendue : Spam
```

Radford et al. (2019, OpenAI) dans *"Language Models are Unsupervised Multitask Learners"* (papier **GPT-2**) montrent les premiers qu'un LLM suffisamment grand peut réaliser des tâches (traduction, résumé, QA…) en zero-shot, sans avoir jamais été explicitement entraîné dessus, simplement en formulant la tâche comme un prompt en langage naturel.

Few-shot prompting

On ajoute quelques **exemples** (*demonstrations*) dans le prompt avant la vraie requête, toujours **sans mise à jour des poids**.

```
Prompt : "Analyse le sentiment (Positif/Négatif/Neutre) :

Avis : 'Livraison rapide, produit conforme, rien à redire.'
Sentiment : Positif

Avis : 'Deux semaines d'attente et l'article était cassé.'
Sentiment : Négatif

Avis : 'Correct pour le prix, sans plus.'
Sentiment :"
```

Brown et al. (2020, OpenAI) dans *"Language Models are Few-Shot Learners"* (**GPT-3**) montrent qu'ajouter des exemples dans le prompt **améliore la performance**, et que ce gain du few-shot **s'accentue à mesure que le modèle grandit** (175B tire bien plus parti des exemples qu'un petit modèle).

Pourquoi l'in-context learning fonctionne-t-il ?

Une question encore débattue en recherche.

Min et al. (2022) dans *"Rethinking the Role of Demonstrations"* : résultat surprenant : remplacer les **bons labels** des exemples par des labels **aléatoires** ne dégrade que peu la performance sur beaucoup de tâches.

→ Le modèle apprend surtout le **format** de la tâche (quel type de sortie produire) et l'**espace des réponses possibles**, pas nécessairement la relation input→output à partir des exemples eux-mêmes.

Xie et al. (2022) proposent une autre lecture : le prompt agirait comme un indice qui aide le modèle à **retrouver**, parmi les tâches implicitement apprises au pré-entraînement, celle qu'on veut lui faire exécuter.

Chain-of-Thought (CoT) prompting

Wei et al. (2022, Google), *"Chain-of-Thought Prompting Elicits Reasoning in LLMs"*.

Idée : demander au modèle d'expliciter son raisonnement **étape par étape** avant de donner la réponse finale, plutôt que de répondre directement.

```
Sans CoT :
Q: Un serveur traite 240 requêtes en 4 secondes. Combien de secondes
   pour traiter 900 requêtes au même débit ?
A: 12                                          ← souvent faux

Avec CoT :
Q: [même question]. Explique ton raisonnement étape par étape.
A: Le débit est 240/4 = 60 requêtes/seconde.
   Pour 900 requêtes : 900/60 = 15 secondes.
   Réponse : 15                                ← correct
```

CoT : zero-shot et self-consistency

Kojima et al. (2022) dans *"LLMs are Zero-Shot Reasoners"* : il suffit d'ajouter **"Let's think step by step"** à la fin du prompt, **sans aucun exemple**, pour déclencher un raisonnement explicite et améliorer nettement les scores sur des benchmarks de raisonnement.

Wang et al. (2022) introduit la *self-consistency* : générer **plusieurs** chaînes de raisonnement (en échantillonnant, pas en greedy) pour la même question, puis garder la réponse finale **majoritaire**.

```
Chaîne 1 → réponse : 15
Chaîne 2 → réponse : 15
Chaîne 3 → réponse : 12
                        → réponse retenue : 15 (majorité)
```

🧩 Quiz — Prompting

1. Quelle est la différence fondamentale entre le fine-tuning et le prompting pour adapter un modèle à une tâche ?

2. Min et al. (2022) remplacent les labels corrects des exemples few-shot par des labels aléatoires. Que révèle ce résultat sur ce que le modèle apprend réellement du prompt ?

3. Que change concrètement le Chain-of-Thought dans la façon dont le modèle produit sa réponse ?

🧩 Quiz — Réponses

1. Le fine-tuning **modifie les poids** du modèle (coûteux, permanent) ; le prompting ne modifie que le **texte d'entrée**, à poids figés (gratuit, réversible, instantané).

2. Le modèle s'appuie surtout sur le **format** de la tâche et l'espace des réponses attendues, pas uniquement sur la relation input → output démontrée dans les exemples.

3. Le modèle génère un raisonnement intermédiaire, token par token, **avant** la réponse finale. Chaque étape de raisonnement conditionne la suivante, ce qui améliore les tâches nécessitant plusieurs étapes de calcul ou de logique.

3. Contraintes d'inférence

Un modèle entraîné, mais il faut le faire tourner

Le pré-entraînement et l'alignement (séance 3) sont des coûts **ponctuels**. L'**inférence**, elle, se répète à **chaque requête**, potentiellement des millions de fois par jour.

Trois contraintes structurent tout déploiement de LLM en production :

- La **fenêtre de contexte** : combien de tokens le modèle peut-il traiter à la fois ?
- La **quantisation** : comment réduire la taille du modèle en mémoire ?
- Le **coût** : combien coûte chaque requête, et comment le réduire ?

Qu'est-ce que l'inférence ?

Inférence = faire passer une entrée dans le modèle **déjà entraîné** pour obtenir une sortie (une suite de tokens) — par opposition à l'**entraînement**, qui ajuste les poids.

```
Entraînement : forward + backward + mise à jour des poids   (une fois)
Inférence    : forward seul, poids figés                    (à chaque requête)
```

- L'inférence tourne, comme l'entraînement, sur des **GPU** (parfois CPU pour de petits modèles) : ce sont des multiplications matricielles massivement parallélisables.
- Pas de rétropropagation ni de stockage des gradients → moins coûteux **par appel** que l'entraînement, mais répété des millions de fois par jour en production.

Ordre de grandeur du coût : `coût ≈ (temps GPU en heures) × (prix horaire du GPU) / (nb de requêtes traitées en parallèle)`. D'où l'intérêt de tout ce qui suit : réduire le temps de calcul (contexte, quantisation) et maximiser le parallélisme (KV-cache, batching).

La fenêtre de contexte

Rappel (séance 2) : l'attention calcule une interaction entre **chaque paire de tokens** → coût **quadratique** en la longueur de séquence n : $O(n^2)$.

Doubler le contexte ≈ quadrupler le calcul d'attention.

Modèle	Fenêtre de contexte (ordre de grandeur)
GPT-3 (2020)	2 048 tokens
GPT-4 (2023)	8k – 128k tokens selon la variante
Claude 3+	~200 000 tokens
Gemini 1.5 (2024)	~1 à 2 millions de tokens (annoncé)

Optimisation du calcul de l'attention

Dao et al. (2022) — *FlashAttention* : recalcule l'attention exacte (pas d'approximation) en minimisant les lectures/écritures mémoire GPU → mêmes résultats, bien plus rapide, permet des contextes plus longs en pratique.

Le nombre d'opérations reste $O(n^2)$ en théorie.
FlashAttention élimine un goulot d'étranglement mémoire (moins d'aller-retours entre la mémoire GPU lente et rapide), d'où le gain de vitesse réel.

Dao (2023) — *FlashAttention-2* : même idée, mais avec un meilleur **partitionnement du travail entre threads/warps du GPU** → jusqu'à 2× plus rapide que la v1, en exploitant mieux le matériel, toujours sans changer la complexité théorique $O(n^2)$.

Quantisation

Idée : stocker les poids du modèle avec **moins de bits** (FP32/FP16 → INT8 → INT4), pour réduire la mémoire et accélérer l'inférence, au prix d'une petite perte de précision numérique.

```
FP16 : 2 octets / paramètre   → LLaMA 65B ≈ 130 Go de RAM/VRAM
INT4 : 0,5 octet / paramètre  → LLaMA 65B ≈  35 Go de RAM/VRAM
```

Dettmers et al. (2022) — *LLM.int8()* : inférence en 8 bits sans perte de performance mesurable, en traitant séparément quelques dimensions à forte magnitude (*outliers*).

Dettmers et al. (2023) — *QLoRA* : quantisation en 4 bits + fine-tuning léger (LoRA) → un modèle de 65 milliards de paramètres fine-tuné sur un **unique GPU de 48 Go**, au lieu de plusieurs dizaines de GPU.

Le coût de l'inférence

Les API de LLM facturent au **token** (entrée + sortie); d'où l'importance de la tokenization vue en séance 3 (fertilité, vocabulaire).

Ordres de grandeur (indicatifs, évoluent vite), prix pour 1 million de tokens :

Modèle	Entrée	Sortie
Modèle frontière (GPT-4 Turbo, Claude Opus)	~10 $	~30-75 $
Modèle frontière "rapide" (GPT-4o, Claude Sonnet)	~2-3 $	~10-15 $
Petit modèle open-source hébergé (LLaMA 8B)	< 0,20 $	< 0,20 $

Côté infrastructure : louer un GPU type A100/H100 sur le cloud coûte de l'ordre de **1 à 4 $/heure** selon le fournisseur et la disponibilité (le prix au token payé par le client couvre ce coût GPU, réparti sur toutes les requêtes traitées en parallèle sur cette même carte).

Optimiser l'inférence : KV-cache

Générer un token nécessite l'attention entre ce token et **tous** les précédents. Sans optimisation, générer le token t referait tout le calcul d'attention sur les $t-1$ tokens précédents. Ceci est **redondant**, puisque ce calcul a déjà été fait pour générer les tokens précédents.

KV-cache : on garde en mémoire les vecteurs **clé (K)** et **valeur (V)** pour chaque token généré. À l'étape suivante, on calcule Q/K/V **seulement pour le nouveau token**, et on réutilise le K/V déjà en cache pour le reste.

```
Sans cache : token t → recalcule K,V pour les t-1 tokens précédents + le nouveau
Avec cache : token t → réutilise K,V en cache, calcule seulement K,V du nouveau token
```

Optimiser l'inférence : KV-cache

Kwon et al. (2023), *PagedAttention / vLLM* : gère la mémoire du KV-cache comme un système d'exploitation gère la RAM (pagination), ce qui permet de servir **beaucoup plus de requêtes en parallèle** avec le même matériel.

- Compromis constant en production : **latence** (répondre vite à une requête) vs **débit** (traiter un grand nombre de requêtes en parallèle).

🧩 Quiz — Contraintes d'inférence

1. Pourquoi doubler la fenêtre de contexte d'un Transformer coûte-t-il plus que deux fois plus cher en calcul d'attention ?

2. Quel est le compromis fondamental de la quantisation ?

3. À quoi sert le KV-cache pendant la génération de texte ?

🧩 Quiz — Réponses

1. L'attention calcule une interaction entre chaque paire de tokens : le coût est **quadratique** ($O(n^2)$) en la longueur de séquence, pas linéaire.

2. Réduire la précision numérique des poids (moins de bits) pour gagner en mémoire et en vitesse, au prix d'une **légère perte de qualité** — souvent négligeable jusqu'à 8 ou 4 bits.

3. Il évite de recalculer les clés/valeurs d'attention des tokens déjà générés à chaque nouveau token — sans lui, chaque token généré recoûterait aussi cher que toute la séquence précédente.

4. Retrieval-Augmented Generation (RAG)

Le problème des connaissances figées

Un LLM ne connaît que ce qu'il a vu au pré-entraînement (séance 3) :

- Ses connaissances s'arrêtent à une **date de coupure** (*knowledge cutoff*)
- Il n'a **aucun accès** aux documents privés d'une entreprise, à une base de code interne, ou à des données mises à jour après son entraînement
- Il peut produire une réponse **incorrecte formulée avec le même style assuré qu'une réponse correcte** — rien dans le texte généré ne signale l'incertitude ou l'absence de connaissance réelle

Ré-entraîner ou fine-tuner le modèle à chaque mise à jour de connaissance serait beaucoup trop coûteux.

Le pipeline RAG

Lewis et al. (2020, Facebook AI) dans *"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks"*.

```
[Corpus de documents]
        ↓ embeddings (séance 1/2)
[Base vectorielle]

[Question utilisateur] → embedding → recherche des k passages
                                     les plus proches (similarité cosinus)
                                           ↓
                          [passages retrouvés + question] → Prompt du LLM
                                           ↓
                                    Réponse générée
```

Le LLM **n'est jamais ré-entraîné** : les documents retrouvés sont simplement injectés comme contexte dans le prompt.

RAG : un exemple concret

Sans RAG, une question sur une politique interne à l'entreprise, absente des données d'entraînement :

```
Q : "Quelle est la limite de requêtes par minute de notre API interne
     `PaymentGateway` ?"
R : "En général, ce type d'API interne impose une limite de l'ordre
     de 100 à 1000 requêtes par minute selon la charge..."
     ← plausible, mais inventé : le modèle n'a jamais vu cette doc
```

Avec RAG, le passage pertinent de la documentation interne est injecté dans le prompt :

```
Contexte injecté : "PaymentGateway applique un quota de 240 requêtes/min
                    par client, avec un burst autorisé de 400 requêtes
                    sur 10 secondes (doc interne, révision 2025-11)."
Q : "Quelle est la limite de requêtes par minute de PaymentGateway ?"
R : "240 requêtes par minute par client, avec un burst possible de
     400 requêtes sur 10 secondes."   ← ancré dans la vraie doc
```

Composants pratiques d'un système RAG

Composant	Rôle	Exemples
Chunking	Découper les documents en passages	taille fixe, par paragraphe, overlap
Embedding model	Vectoriser chunks et requêtes	`sentence-transformers` (ecodeur)
Base vectorielle	Stocker et rechercher par similarité	FAISS (Johnson et al., 2019), Pinecone, Chroma
Génération	Produire la réponse finale	n'importe quel LLM (via prompt)

FAISS (*Facebook AI Similarity Search*) permet de rechercher les voisins les plus proches parmi des **milliards** de vecteurs efficacement (recherche approximative sur GPU).

Limites du RAG

- La retrieval peut **échouer** : passages non pertinents récupérés → réponse basée sur du bruit
- La fenêtre de contexte limite **combien** de passages on peut injecter (section 3)
- Le modèle peut encore **halluciner**, même avec un contexte pertinent fourni (il n'est pas obligé de s'y tenir)
- Le RAG apporte des **connaissances**, pas de nouvelles **compétences** : il ne remplace pas un fine-tuning pour changer le style ou le comportement du modèle

🧩 Quiz — RAG

1. Pourquoi le RAG évite-t-il d'avoir à ré-entraîner le modèle à chaque mise à jour des données ?

2. Quel rôle jouent les embeddings (séance 1/2) dans un système RAG ?

3. Citez une limite du RAG qui persiste même quand la retrieval fonctionne bien.

🧩 Quiz — Réponses

1. Les documents à jour sont récupérés **à la volée** et injectés dans le prompt au moment de la requête — le modèle lui-même reste figé, seul le contenu du prompt change.

2. Ils permettent de représenter documents et requêtes comme des vecteurs, pour retrouver par **similarité** les passages les plus pertinents à injecter dans le prompt.

3. Le modèle peut halluciner malgré un contexte pertinent fourni ; ou la fenêtre de contexte limite le nombre de passages injectables.

5. Appel d'outils et function calling

Ce qu'un LLM ne peut pas savoir tout seul

- L'heure ou la météo **actuelles**
- Un calcul **précis** sur de grands nombres (les LLMs approximent souvent l'arithmétique)
- L'état d'une base de données ou d'un système **externe**
- Exécuter réellement du **code**

Solution : laisser le modèle **appeler des outils externes** plutôt que de tout générer lui-même à partir de ses poids.

Function calling : le mécanisme

Au lieu de générer uniquement du texte, le modèle peut produire un **appel structuré** :

```
Utilisateur : "Quel temps fait-il à Paris demain ?"

Modèle → { "function": "get_weather",
           "arguments": {"city": "Paris", "date": "2026-07-11"} }

Application → exécute réellement la fonction, récupère le résultat
           → { "temperature": 22, "condition": "nuageux" }

Modèle → "Il devrait faire environ 22°C et nuageux à Paris demain."
```

Le modèle **décide quand** appeler un outil et **avec quels arguments**.

Toolformer : apprendre à s'en servir seul

Schick et al. (2023, Meta AI) dans *"Toolformer: Language Models Can Teach Themselves to Use Tools"*.

Problème : annoter manuellement des millions d'exemples "quand appeler quel outil" serait extrêmement coûteux.

Méthode auto-supervisée :

```
1. Le modèle génère lui-même des appels d'API candidats, insérés
   dans du texte brut (calculatrice, QA, recherche, calendrier, traduction...)
2. Les appels sont réellement exécutés → on obtient un vrai résultat
3. On compare la perplexité du modèle sur les tokens qui suivent, avec vs.
   sans le résultat de l'appel inséré dans le texte
4. On ne garde l'exemple que si le résultat réduit la perplexité d'une
   marge minimale, sinon l'appel est jeté (jugé inutile)
5. Le modèle est fine-tuné (SFT classique) sur les exemples filtrés
```

Toolformer : apprendre à s'en servir seul

La perplexité : c'est l'exponentielle de la perte d'entraînement. Elle mesure à quel point le modèle est "surpris" par la suite réelle du texte : plus la perplexité est **basse**, plus le modèle attribuait une **probabilité élevée** aux tokens qui suivent réellement, donc le résultat de l'appel d'outil rend le texte suivant plus prévisible pour le modèle.

Aucune annotation humaine : le modèle apprend **seul** quand un outil est utile.

Des outils aux agents

LLM + outils + boucle de décision = agent.

- Les APIs modernes (OpenAI, Anthropic, Mistral…) exposent nativement du *tool use / function calling*
- Le **Model Context Protocol (MCP)**, ouvert par Anthropic en 2024, standardise la façon dont un modèle découvre et appelle des outils externes
- Function calling = la **brique de base** ; comment un LLM enchaîne plusieurs appels, raisonne sur leurs résultats et se coordonne avec d'autres agents → **séance 5** (architectures agentiques, ReAct, systèmes multi-agents)

🧩 Quiz — Function calling

1. Qui exécute réellement la fonction lors d'un function calling : le LLM ou l'application appelante ?

2. Comment Toolformer décide-t-il si un appel d'outil candidat est utile, sans supervision humaine ?

3. Pourquoi ne peut-on pas se contenter de fine-tuner un LLM pour qu'il "sache" la météo ou l'heure actuelle ?

🧩 Quiz — Réponses

1. L'**application appelante** : le LLM ne fait que décider quand appeler un outil et avec quels arguments ; il ne l'exécute jamais lui-même.

2. Il garde uniquement les appels qui **réduisent la perplexité** du modèle sur la suite du texte, par rapport à ne pas faire d'appel = un critère purement auto-supervisé.

3. Les poids du modèle sont **figés** après l'entraînement (séance 3) : une information qui change en continu (météo, heure, cours de bourse…) ne peut pas être "sue" par des poids fixes — elle doit être récupérée **à la volée** via un outil.

Récapitulatif : de l'entraînement à l'usage

```
Modèle pré-entraîné, instruction-tuné, aligné (séance 3)
    ↓  prompting (zero-shot, few-shot, Chain-of-Thought)
Comportement adapté à la tâche, sans ré-entraînement
    ↓  contraintes d'inférence (contexte, quantisation, coût)
Modèle déployé efficacement en production
    ↓  RAG
Réponses ancrées dans des connaissances à jour / privées
    ↓  function calling (Toolformer)
Modèle capable d'agir sur le monde extérieur → agent (suite en séance 5)
```

Ressources

- **GPT-2 (zero-shot)** : Radford et al. (2019) : "Language Models are Unsupervised Multitask Learners"
- **GPT-3 (few-shot)** : Brown et al. (2020); arxiv.org/abs/2005.14165
- **Contamination des benchmarks** : Oren et al. (2023); arxiv.org/abs/2310.17623
- **Chain-of-Thought** : Wei et al. (2022); arxiv.org/abs/2201.11903
- **Zero-shot CoT** : Kojima et al. (2022); arxiv.org/abs/2205.11916
- **Self-Consistency** : Wang et al. (2022); arxiv.org/abs/2203.11171
- **Rethinking Demonstrations** : Min et al. (2022); arxiv.org/abs/2202.12837
- **FlashAttention** : Dao et al. (2022); arxiv.org/abs/2205.14135
- **FlashAttention-2** : Dao (2023); arxiv.org/abs/2307.08691
- **LLM.int8()** : Dettmers et al. (2022); arxiv.org/abs/2208.07339
- **QLoRA** : Dettmers et al. (2023); arxiv.org/abs/2305.14314
- **vLLM / PagedAttention** : Kwon et al. (2023); arxiv.org/abs/2309.06180
- **RAG** : Lewis et al. (2020); arxiv.org/abs/2005.11401
- **Toolformer** : Schick et al. (2023); arxiv.org/abs/2302.04761

Lab 4 : Prompting et RAG (1h-2h)

Partie 1 : Prompting comparatif

Sur un même petit jeu de problèmes (logique/arithmétique), comparez zero-shot, few-shot et Chain-of-Thought avec un modèle Hugging Face. Mesurez l'exactitude de chaque approche.

Partie 2 : Mini-RAG

Construisez un pipeline RAG simple : indexez quelques documents avec `sentence-transformers` (embeddings, cf. lab 2) + `faiss`, récupérez les passages pertinents pour une question, injectez-les dans le prompt d'un LLM et comparez la réponse avec/sans retrieval.